

LangGraph overview

 Copy page

Trusted by companies shaping the future of agents— including Klarna, Uber, J.P. Morgan, and more — LangGraph is a low-level orchestration framework and runtime for building, managing, and deploying long-running, stateful agents.

LangGraph is very low-level, and focused entirely on agent **orchestration**. Before using LangGraph, we recommend you familiarize yourself with some of the components used to build agents, starting with [models](#) and [tools](#).

We will commonly use [LangChain](#) components throughout the documentation to integrate models and tools, but you don't need to use LangChain to use LangGraph. If you are just getting started with agents or want a higher-level abstraction, we recommend you use LangChain's [agents](#) that provide prebuilt architectures for common LLM and tool-calling loops.

LangGraph is focused on the underlying capabilities important for agent orchestration: durable execution, streaming, human-in-the-loop, and more.

> Show how LangChain products fit together

Install

`pip` `uv`

```
pip install -U langgraph
```



Then, create a simple hello world example:

```
graph = StateGraph(MessagesState)
graph.add_node(mock_llm)
graph.add_edge(START, "mock_llm")
graph.add_edge("mock_llm", END)
graph = graph.compile()

graph.invoke({"messages": [{"role": "user", "content": "hi!"}]})
```

💡 Use [LangSmith](#) to trace requests, debug agent behavior, and evaluate outputs. Set `LANGSMITH_TRACING=true` and your API key to get started. Follow the [tracing quickstart](#) to get set up. We recommend you also set up [LangSmith Engine](#) which monitors your traces, detects issues, and proposes fixes.

Core benefits

LangGraph provides low-level supporting infrastructure for *any* long-running, stateful workflow or agent. LangGraph does not abstract prompts or architecture, and provides the following central benefits:

- Persistence: Build agents that persist through failures and can run for extended periods, resuming from where they left off.
- Human-in-the-loop: Incorporate human oversight by inspecting and modifying agent state at any point.
- Comprehensive memory: Create stateful agents with both short-term working memory for ongoing reasoning and long-term memory across sessions.
- Debugging with LangSmith: Gain deep visibility into complex agent behavior with visualization tools that trace execution paths, capture state transitions, and provide detailed runtime metrics.
- Production-ready deployment: Deploy sophisticated agent systems confidently with scalable infrastructure designed to handle the unique challenges of stateful, long-running workflows.

LangGraph ecosystem



LangSmith Observability

Trace requests, evaluate outputs, and monitor deployments in one place. Prototype locally with LangGraph, then move to production with integrated observability and evaluation to build more reliable agent systems.

[Learn more >](#)



LangSmith Deployment

Deploy and scale agents effortlessly with a purpose-built deployment platform for long running, stateful workflows. Discover, reuse, configure, and share agents across teams — and iterate quickly with visual prototyping in Studio.

[Learn more >](#)



LangChain

Provides integrations and composable components to streamline LLM application development. Contains agent abstractions built on top of LangGraph.

[Learn more >](#)

Acknowledgements

LangGraph is inspired by [Pregel](#) and [Apache Beam](#). The public interface draws inspiration from [NetworkX](#). LangGraph is built by LangChain Inc, the creators of LangChain, but can be used without LangChain.

Was this page helpful?

 Yes

 No